SRM TRP ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY
TIRUCHIRAPPALLI

AUTONOMOUS

Department of Artificial Intelligence&Data Science

CRICKET SCORE PREDICTION: LITERATURE SURVEY, RESEARCH GAP ANALYSIS AND A CONTEXT-AWARE DEEP LEARNING FRAMEWORK

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE&DATA SCIENCE

Submitted By

BOOBESHWARAN P – 814724243025

DHAKEER KHAN S – 814724243032

BHUVANESHWAR R – 814724243024

BALACHANDRAN R – 814724243021

Under the Guidance of

Mrs. R. Kalaiyarasi

SRM TRP ENGINEERING COLLEGE

Academic Year: 2026-27 (ODD)

SRM TRP ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY
TIRUCHIRAPPALLI

AUTONOMOUS

Department of Artificial Intelligence&Data Science

CERTIFICATE

This is to certify that BOOBESHWARAN P (814724243025), DHAKEER KHAN S (814724243032), BHUVANESHWAR R (814724243024) AND BALACHANDRAN R (814724243021) from **Third year AI&DS** has successfully completed the work titled **"Cricket Score Prediction: Literature Survey, Research Gap Analysis and a Context-Aware Deep Learning Framework"** at SRM TRP Engineering College, Tiruchirappalli, in the partial fulfilment of the Bachelor of Technology in Artificial Intelligence & Data Science.

Mrs. R. Kalaiyarasi
(Faculty In-charge)

Artificial Intelligence&Data Science

Dr. R. Perumal Raja

(Head of Department)
Artificial Intelligence&Data Science

ACKNOWLEDGEMENT

It is with sincere gratitude and deep appreciation that we take this opportunity to acknowledge the invaluable support and guidance we have received throughout the course of this project. This journey has been both intellectually enriching and personally rewarding, significantly enhancing our understanding of sports analytics, deep learning for sequential data, and the practical challenges of forecasting in a game as uncertain as cricket.

We would like to express our heartfelt thanks to our faculty in-charge, Mrs. R. Kalaiyarasi, and to the Head of the Department, Dr. R. Perumal Raja, for their guidance, encouragement, and constructive feedback during the development of this project. Their insights were instrumental in shaping the direction and quality of the work. We also extend our appreciation to the Department of Artificial Intelligence & Data Science for fostering an environment that promotes learning, innovation, and academic growth. The resources and opportunities provided played a vital role in the successful completion of this project.

A special note of thanks goes to the supporting staff for their cooperation and timely assistance, which ensured that the process remained smooth and efficient at every stage.

Finally, we express our deepest gratitude to our parents for their constant support, encouragement, and belief in our abilities. This project has not only strengthened our knowledge but has also emphasized the importance of dedication, perseverance, and continuous learning. We remain sincerely thankful to everyone who contributed, directly or indirectly, to the successful completion of this project.

Academic Year:

2026-2027

BOOBESHWARAN P – 814724243025

DHAKEER KHAN S – 814724243032

BHUVANESHWAR R – 814724243024

BALACHANDRAN R – 814724243021

TABLE OF CONTENTS

Sr. No	Title	Page No.
1	Acknowledgement	3
2	Abstract	5
3	Introduction	6
4	System Requirements	8
5	Literature Survey	9
6	Research Gap Identification	13
7	Methodology	15
8	Implementation	19
9	Mapping to Sustainable Development Goal 9	21
10	Conclusion and Future Work	22
11	References	22

ABSTRACT

Cricket is among the most data-rich team sports in the world. Every delivery produces a record of runs, wickets, bowler, batter and match situation, and franchise leagues such as the Indian Premier League (IPL) have generated more than a decade of ball-by-ball data. Predicting the final score of an innings while the match is in progress is valuable to teams planning their batting and bowling strategy, to broadcasters presenting live win and score projections, and to analysts. The traditional approach, projecting the final score from the current run rate, is simple but ignores wickets in hand, the phase of the innings, the venue and the strength of the two sides.

This project presents a literature survey and research gap analysis on cricket score prediction. The foundational work studied is “Sports Analytics in Cricket – Predicting Scores in the Indian Premier League Using Deep Learning Techniques” published in the ICTACT Journal on Data Science and Machine Learning (2025). It trains Feedforward Neural Networks, Multilayer Perceptrons, Recurrent Neural Networks and Long Short-Term Memory (LSTM) networks on ten seasons of ball-by-ball IPL data and reports that the LSTM gives the best accuracy, with an 85% success rate. The survey further compares nine additional studies covering data-mining approaches to score and winning prediction, machine learning with Random Forest and related classifiers, deep learning for first-innings score forecasting, Duckworth-Lewis resource tables and their comparison with machine learning, and IPL match outcome prediction.

The analysis identifies a reliability gap. Most existing systems output a single number, are evaluated only on the final score or on overall accuracy, and say little about how the error changes as the innings unfolds or how uncertain the forecast is. Many also depend on static pre-match features or a run-rate formulation that cannot react to the situation of the innings. Based on this gap, the project proposes a context-aware deep learning framework that encodes the last few overs with a recurrent network, fuses them with venue, team and resource features, and outputs both an expected final score and a prediction interval. The work is mapped to United Nations Sustainable Development Goal 9 (Industry, Innovation and Infrastructure), since affordable and transparent analytics strengthens technological capability in the sports-technology industry.

Keywords: Deep Learning, Cricket Score Prediction, LSTM, Sports Analytics, IPL, Time-Series Forecasting, SDG 9.

INTRODUCTION

1.1 Background

Cricket is played in three major formats, Test matches, One Day Internationals (ODI) and Twenty20 (T20), and is followed by hundreds of millions of people. The shorter formats, and in particular the IPL, have made cricket a heavily data-driven sport. Every ball is recorded with the batter, bowler, runs, extras and dismissals, so that a single season yields tens of thousands of structured records. Teams use this data for selection and match strategy, broadcasters use it for live graphics, and fans follow projected scores and win probabilities as the game unfolds.

The central difficulty is that cricket is a sequential game with many interacting factors. The final score of an innings depends on how many wickets are in hand, how many overs remain, how strong the batting and bowling units are, how the pitch and venue behave, and how the innings has progressed over time. A team on 60 for 0 after eight overs and a team on 60 for 4 have the same score but very different prospects. A prediction method that cannot represent this difference will be wrong in exactly the situations where an accurate forecast matters most.

1.2 What is Cricket Score Prediction?

Cricket score prediction is the task of estimating the total runs a team will score in its innings, either before the innings starts or at any point during it. The studies reviewed in this project treat it in three related ways [1]–[10]:

- **Pre-match score prediction:** estimating the first-innings total from teams, venue and historical form, before a ball is bowled.
- **In-play (live) score prediction:** updating the forecast of the final score after each ball or over using the current match state, such as score, wickets and overs completed.
- **Outcome prediction:** predicting which team will win, often using the first-innings score as a key input [2], [8].

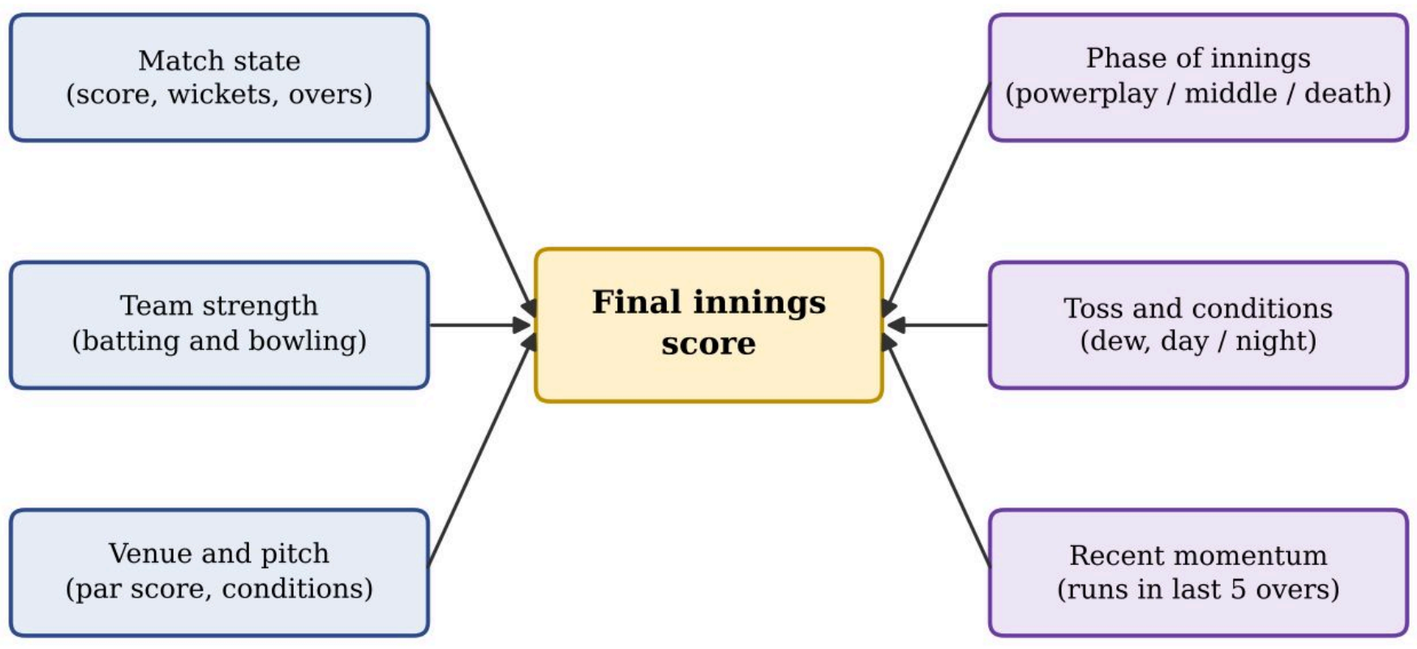


Figure 1.1: Factors that influence the final score of an innings

1.3 Why Score Prediction Matters

Accurate score prediction has practical value for several groups. Coaches and captains use projected totals to decide when to accelerate, how to set a bowling plan

and what target is defensible. Broadcasters present live projections that make a match easier to follow. Analysts use predictions to compare how a team performs against the expectation set by the situation. The same models can also serve as a neutral benchmark of what a par score is at a given venue and stage of the innings.

Score prediction also has a methodological importance. It is a clean, well-defined sequential forecasting problem with abundant public data, which makes it a good test case for time-series deep learning. Lessons about how to represent match state, how to avoid data leakage and how to report uncertainty carry over to other forecasting domains.

1.4 Problem Statement

The simplest and most widely used method, projecting the final score from the current run rate, is easy to compute but ignores wickets in hand, the phase of the innings and the venue. Classical machine learning models improve on it but often rely on hand-picked aggregate features and report only a single accuracy figure. Deep learning models such as the LSTM reported in [1] give better accuracy, yet they typically return a single point estimate, do not show how the error evolves as the innings progresses, and are rarely evaluated across stages of the match. There is a need for a framework that predicts the final score from the live match state, is accurate at every stage of the innings, and states how uncertain the forecast is.

1.5 Objectives

1. To study the foundational deep learning approach to IPL score prediction and understand its strengths and limitations.
2. To conduct a comparative literature survey of ten studies on score prediction, resource-based methods and match outcome prediction.
3. To identify a clear research gap in the accuracy, uncertainty reporting and evaluation of current score prediction techniques.
4. To propose a context-aware deep learning architecture that addresses the identified gap.
5. To map the contribution of the work to United Nations Sustainable Development Goal 9.

1.6 Scope of the Work

This report focuses on predicting the final score of the team batting first in a T20 innings, with emphasis on IPL-style ball-by-ball data. Related areas such as match winner prediction, player performance and fantasy-points prediction, and interrupted-match target setting are discussed where they inform the survey, but the central contribution is the analysis of score prediction methods and the design of a framework that is accurate across stages of the innings and reports uncertainty.

1.7 Organisation of the Report

Chapter 2 lists the hardware and software requirements. Chapter 3 presents the literature survey, including a comparative analysis of ten papers. Chapter 4 identifies the research gap. Chapter 5 describes the methodology, covering the formal problem

definition, the baseline and the proposed framework. Chapter 6 describes the implementation plan, algorithms and an illustrative worked example. Chapter 7 maps the work to SDG 9, and Chapter 8 concludes with future directions.

SYSTEM REQUIREMENTS

The tools below are recommended for implementing and evaluating the experiments described in this report, including the run-rate and resource baselines and the proposed sequence model.

Category	Requirement	Description
Operating System	Windows / Linux / macOS	Any OS supporting Python and deep learning frameworks. Linux is preferred for GPU drivers and better performance.
Processor (CPU)	Intel i5/i7 or AMD Ryzen 5/7 (or higher)	Multi-core processors speed up data loading, feature engineering and baseline model training.
GPU (Optional)	NVIDIA GPU with 6 GB+ VRAM (e.g., GTX 1660 or RTX 3050)	Speeds up training of the sequence model with CUDA. The proposed model is small, so training on a CPU or a free cloud GPU is also feasible.
RAM	Minimum 8 GB (Recommended: 16 GB)	Required for holding ball-by-ball data, over-level feature tables and model batches.
Storage	Minimum 10 GB free space	For raw match files, processed datasets, model checkpoints and experiment logs.
Programming Language	Python 3.10+	Standard language for data science and deep learning with extensive library support.
Deep Learning Framework	PyTorch (or TensorFlow / Keras)	Used to build and train the GRU/LSTM encoder, embeddings and the quantile prediction head.
Machine Learning Libraries	scikit-learn, XGBoost	Provide the Linear Regression, Random Forest and gradient boosting baselines and the evaluation metrics.
Data Handling Libraries	NumPy, Pandas	Used to parse ball-by-ball files, aggregate to over level and build the feature tables.
Visualization Tools	Matplotlib, Seaborn	For error-versus-over curves, prediction intervals and result plots.
Development Environment	Jupyter Notebook / Visual Studio Code	For writing, testing and debugging experiments.
Deployment (Optional)	Streamlit or Flask	A simple web interface where the user enters the current match state and receives the predicted score and interval.
Datasets	Cricsheet ball-by-ball data; IPL match datasets (e.g., Kaggle)	Provide ball-by-ball records of IPL matches with teams, venues, runs and wickets for training and evaluation.

LITERATURE SURVEY

This chapter reviews ten studies on score and outcome prediction in cricket. The foundational paper, on deep learning for IPL score prediction, is examined in depth; the remaining papers are grouped into early data-mining and machine learning approaches, deep learning for first-innings forecasting, resource-based methods, and match outcome prediction.

3.1 Foundational Paper: Deep Learning for IPL Score Prediction [1]

Raghav, Singh and Kumar published "Sports Analytics in Cricket – Predicting Scores in the Indian Premier League Using Deep Learning Techniques" in the ICTACT Journal on Data Science and Machine Learning in 2025. The study addresses the difficulty of identifying patterns in the large volume of data generated by the IPL, an annual T20 league, and uses the most significant factors to predict the final cumulative score of a team in its innings.

The authors apply four families of deep learning models: Feedforward Neural Networks (FNN), Multilayer Perceptrons (MLP), Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks. The dataset covers ten years of ball-by-ball data from sources such as ESPN and Cricsheet, spanning ten IPL seasons. Models are evaluated with Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and the R^2 value. Among the deep models, the LSTM achieves the highest prediction accuracy, with an 85% success rate.

The result is important for this project for two reasons. First, it confirms that sequence-aware models are better suited than static feedforward models to a game whose state evolves ball by ball, because an LSTM can carry information about how the innings has developed. Second, it shows that a small, well-curated public dataset is sufficient to train such a model. The study is restricted to IPL data, and, like most work in this area, it reports a single prediction for the final score rather than an interval, which motivates the research gap in Chapter 4.

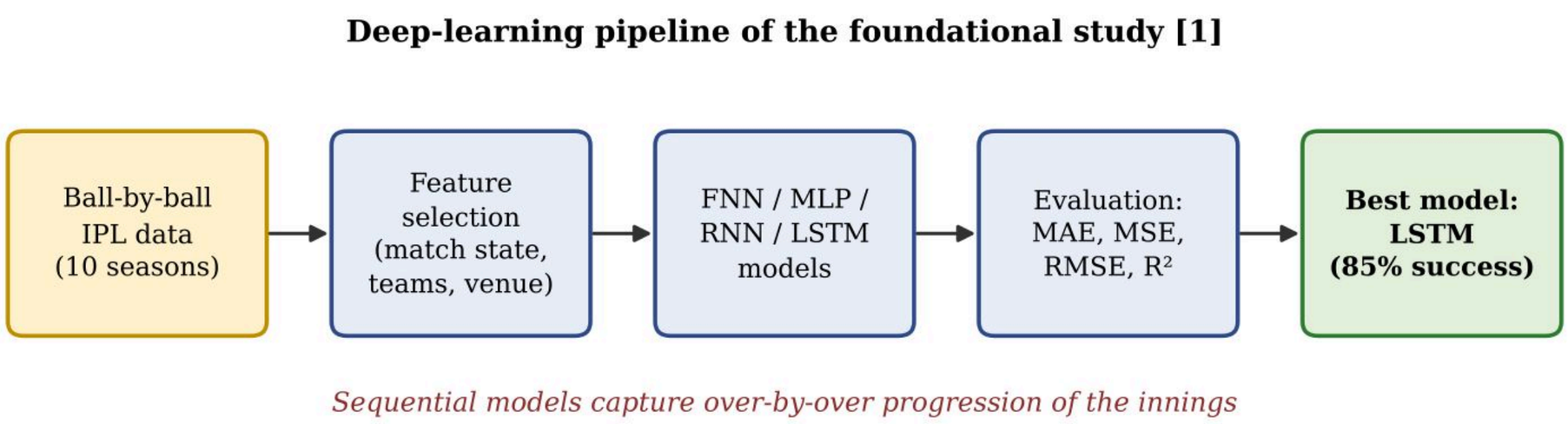


Figure 3.1: Deep learning pipeline of the foundational study [1]

3.2 Data Mining and Machine Learning for Score Prediction [2], [3], [4]

Singh, Singla and Bhatia [2] proposed one of the early data-mining models for cricket. In One Day Internationals, the first-innings score was traditionally predicted from the current run rate. Their model extends this by also using the number of wickets fallen, the venue and the batting team. It uses Linear Regression to predict the

first-innings score and a Naive Bayes classifier to predict the winner after the first innings. The work established that adding match context to the run rate improves a score prediction, but it is limited to ODI matches.

Kamble et al. [3] present a Cricket Score Prediction system that uses machine learning to predict both scores and match outcomes, drawing on historical match data and factors such as the toss decision. Several surveys of the field report that Random Forest models perform strongly in this setting, with accuracy around 91% for [3]. The emphasis is on classification-style accuracy; how the error in runs changes during the innings is not the focus.

Suguna et al. [4] study score prediction together with player categorisation in the 2023 IEEE MysuruCon conference. They note that the final score of the first innings is commonly predicted using the Current Run Rate (CRR) and apply regression and classification techniques to predict scores and classify players by their role in the squad. The study shows the value of machine learning for performance analysis but treats score prediction mainly as a regression problem on aggregated features.

3.3 Deep Learning for First-Innings Forecasting [5]

Abeysuriya, Fernando and Navarathna [5] present “Beyond the Run-rate: Forecasting Framework for First Innings Score in T20 Cricket” at the 2023 Moratuwa Engineering Research Conference. They propose a generalised deep neural network based method to predict the first-innings score of a T20 international match, motivated by the observation that the format has changed substantially and that thousands of matches are now available as data. By moving beyond the run rate, the approach shows the benefit of letting a network learn the interaction of match features, though it focuses on T20 internationals rather than franchise leagues and reports a point forecast.

3.4 Resource-Based Methods and Their Comparison with Machine Learning [6], [7]

The Duckworth-Lewis method models the runs a team can still score as a function of two resources, overs remaining and wickets in hand. It is used to reset targets in interrupted matches and is a natural way to think about how the value of an over depends on the wickets left. Bhattacharya, Gill and Swartz [6] review the Duckworth-Lewis table in the context of Twenty20 cricket and build an alternative resource table from observed scoring rates in international T20 matches. Because T20 batting is more aggressive than one-day batting, they construct a non-parametric table using a Gibbs sampling scheme related to isotonic regression, so that the table remains monotonic in both overs and wickets.

Abbas and Haider [7] compare the Duckworth-Lewis-Stern (DLS) method with several supervised learning algorithms for predicting the result of ODI matches during the second innings. They also optimise the DLS resource table using Particle Swarm Optimization; for example, at the 10th over, on 1,514 samples, the modified table reaches an accuracy of 76.15% against 73.38% for the existing table. The result suggests that there is still room to improve resource values, and that data-driven models are competitive with the classical method. A limitation of resource tables is

that they are generic and do not consider the teams, the players or the venue.

3.5 IPL Match Outcome Prediction [8], [9], [10]

Srikantaiah et al. [8] predict IPL match outcomes with SVM, Random Forest, Logistic Regression and K-Nearest Neighbour using team composition, the batting and bowling averages of each player, past success of the team, toss, venue and day or night play. The Random Forest classifier performs best, with an accuracy of 88.10%.

Sanjaykumar et al. [9] apply Random Forest analysis to predict team success in the IPL 2024 season using player statistics such as batting and bowling averages and fielding contributions. The model reports an MSE of 8.2174, an RMSE of 2.8666 and an R^2 of 0.9173. Menon et al. [10] compare Naive Bayes, Random Forest, Logistic Regression, XGBoost, AdaBoost and Decision Tree for IPL prediction, adding team strength and individual player strength to conventional features such as toss, home ground, weather and pitch conditions.

These three studies show that tree-based ensembles are strong on tabular pre-match features. They predict the result of a match rather than the score, and they use static features that are fixed before play starts, so they cannot update their forecast as an innings develops.

3.6 Comparative Analysis

Table 3.1 consolidates the ten studies by method, advantages and limitations.

Sl. No	Paper / Reference	Venue & Year	Method / Technology	Advantages	Limitations
1	Sports Analytics in Cricket – Predicting Scores in the IPL Using Deep Learning [1]	ICTACT J. Data Sci. Mach. Learn., 2025	FNN, MLP, RNN and LSTM on ten IPL seasons of ball-by-ball data	Sequence model (LSTM) gives the best accuracy, 85% success rate	IPL data only; single point forecast with no prediction interval
2	Score and winning prediction in cricket through data mining [2]	ICSCTI, 2015	Linear Regression (score) and Naive Bayes (winner) using run rate, wickets, venue and team	Goes beyond the run rate by adding match context	ODI only; simple models with limited features
3	Cricket Score Prediction Using Machine Learning [3]	Turk. J. Comput. Math. Educ., 2021	ML models for scores and outcomes using historical data and toss	Random Forest performs strongly (about 91% reported)	Accuracy-based evaluation; error in runs by stage not studied
4	Utilizing ML for Sport Data Analytics in Cricket: Score Prediction and Player Categorization [4]	IEEE MysuruCon, 2023	Regression and classification for scores and player roles; CRR-based formulation	Combines score prediction with player role classification	Aggregated features; no explicit modelling of how the innings unfolds
5	Beyond the Run-rate: Forecasting Framework for First Innings Score in T20 Cricket [5]	IEEE MERCon, 2023	Generalised deep neural network for first-innings T20 score	Moves beyond run rate; learns feature interactions	T20 internationals only; point forecast
6	Duckworth–Lewis and Twenty20 cricket [6]	J. Oper. Res. Soc., 2011	Non-parametric T20 resource table via Gibbs sampling and isotonic regression	Monotonic table that reflects T20 scoring rates	Generic table; ignores teams, players and venue
7	Duckworth-Lewis-Stern method comparison with machine learning approach [7]	Conf. paper, 2019	DLS vs supervised learning; PSO-optimised resource table	Shows data-driven models can match or beat DLS	ODI second-innings result prediction, not score forecasting
8	Prediction of IPL Match Outcome Using ML Techniques [8]	ICHC / arXiv, 2021	SVM, Random Forest, Logistic Regression and KNN on team and player features	Random Forest reaches 88.10% accuracy	Predicts winner, not runs; static pre-match features

Sl. No	Paper / Reference	Venue & Year	Method / Technology	Advantages	Limitations
9	Predicting Team Success in the IPL 2024 Season Using Random Forest Analysis [9]	Phys. Educ. Theory Methodol., 2024	Random Forest on player statistics (MSE 8.2174, RMSE 2.8666, R^2 0.9173)	High explained variance on aggregated player data	Season-level team success; not live score forecasting
10	IPL Prediction Using Machine Learning [10]	Indian J. Comput. Sci., 2022	Naive Bayes, RF, LR, XGBoost, AdaBoost and Decision Tree with team and player strength	Compares many algorithms with several context features	Pre-match features; no ball-by-ball updating

Table 3.1: Comparative analysis of cricket score and outcome prediction literature

Note: advantages and limitations are the team's assessment against this report's objectives, based on the papers' abstracts and summaries.

3.7 Observations from the Survey

- Early and classical models [2]–[4] improve on the run rate by adding wickets, venue and team, but they rely on aggregated features and mostly report accuracy.
- Deep sequence models [1], [5] give the best results, with the LSTM leading in [1], which supports the use of a recurrent encoder over the innings.
- Resource-based methods [6], [7] capture the interaction of overs and wickets elegantly, but the tables are generic and need to be re-fitted as batting styles change.
- Outcome prediction studies [8]–[10] depend on static pre-match features and cannot update during play.
- Uncertainty is under-addressed: none of the surveyed studies reports a prediction interval or evaluates how the error changes through the innings.

RESEARCH GAP IDENTIFICATION

On reviewing the literature, a research gap concerning the reliability of in-play score prediction was identified. Two related limitations emerge, both centred on how current methods use the match state and report their results.

4.1 Gap 1: Static or Context-Poor Forecasts

The run-rate method projects the final score as the current run rate multiplied by the number of overs, which assumes that the scoring rate stays constant. In reality the rate changes with the phase of the innings: batters are restricted in the powerplay, consolidate in the middle overs and accelerate in the death overs. The effect is shown in Figure 4.1, where the run-rate projection moves up and down by more than thirty runs during one illustrative innings while the eventual score stays fixed. Pre-match models [8]–[10] avoid this by not updating at all, so they also cannot use the information that arrives as the innings progresses.

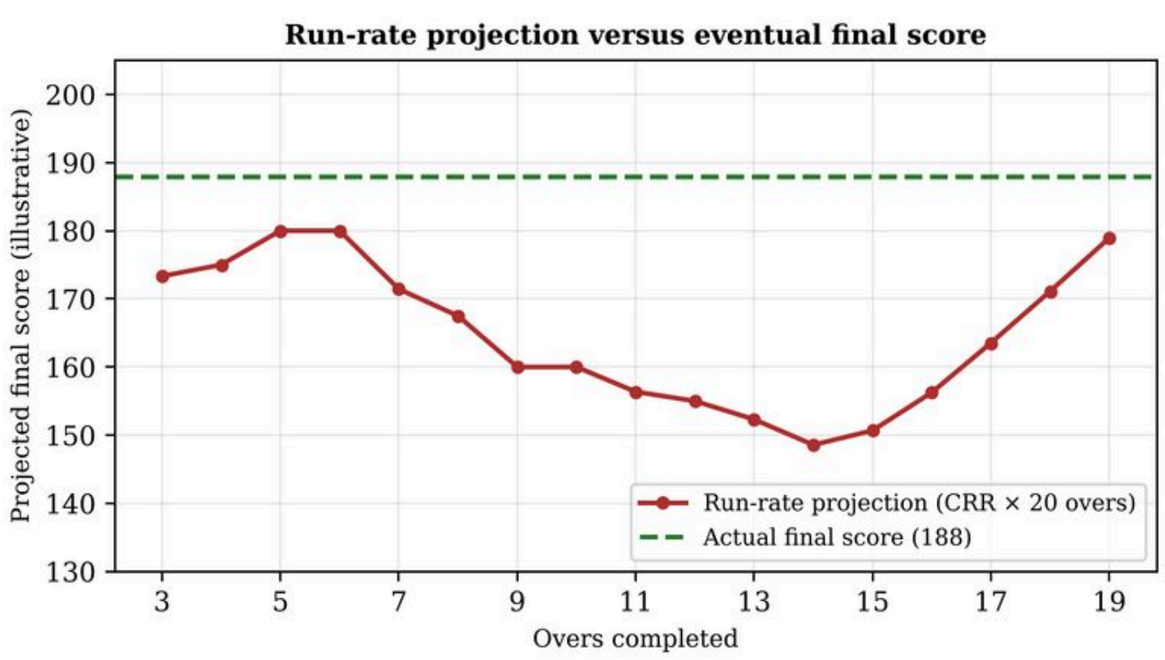


Figure 4.1: Illustrative gap between run-rate projection and the eventual final score

4.2 Gap 2: Point Forecasts and Stage-Blind Evaluation

Even the best deep learning model reported in the survey [1] returns a single number and is judged by overall error or accuracy. A forecast made after five overs is inherently less certain than one made after fifteen overs, yet a single number hides this difference. Users such as captains and broadcasters need to know whether a projected score of 175 means “between 170 and 180” or “between 150 and 200”. Without an interval, and without reporting error separately at each stage of the innings, it is hard to know when the forecast can be trusted.

4.3 Summary of the Gap and Research Question

Aspect	Existing approach	Desired capability
Use of match state	Run rate, aggregate features, or static pre-match data	Live match state encoded over the last few overs
Wickets and phase	Ignored by the run-rate method; generic in resource tables	Explicit wickets-in-hand, overs-left and phase features
Output	Single point estimate	Expected score with a prediction interval (P10 to P90)

Aspect	Existing approach	Desired capability
Evaluation	Overall accuracy or error	Error reported at overs 5, 10 and 15 and interval coverage
Model cost	Varies; deep models need large data	Small model that trains on public data and runs in real time

The research question that this project follows is therefore: **Is it possible to design a lightweight deep learning framework that predicts the final score of a T20 innings from the live match state, stays accurate across the powerplay, middle and death overs, and reports a calibrated prediction interval alongside the expected score?**

METHODOLOGY

The methodology has two parts. The first formalises the prediction problem and the run-rate and resource baselines, which serve as the reference. The second describes the proposed context-aware framework designed to close the gaps identified in Chapter 4. The overall process follows a structured pipeline of data collection, feature extraction, model design, training, evaluation and deployment.

5.1 Formalising the Prediction Problem

Consider the innings of the team batting first in a T20 match. After t overs, let S_t be the runs scored, w_t the wickets lost and x_t the full match state, which includes the over-by-over history, the teams and the venue. The task is to predict the final score Y of the innings. The run-rate baseline is

$$\hat{Y}_{CRR} = (S_t / t) \times 20$$

A resource-based baseline uses the idea of [6], [7]. If $R(t, w_t)$ is the share of the innings resources that remain, then the runs scored so far correspond to the share already used, and

$$\hat{Y}_{RES} = S_t / (1 - R(t, w_t))$$

The proposed model replaces these fixed formulae with a learned function $\hat{Y} = f_\theta(x_t)$ that also produces a lower and an upper quantile. The error of a point forecast over N matches is measured by the Mean Absolute Error and the Root Mean Squared Error:

$$MAE = (1/N) \Sigma |Y_i - \hat{Y}_i| , \quad RMSE = \sqrt{[(1/N) \Sigma (Y_i - \hat{Y}_i)^2]}$$

For the prediction interval, the pinball (quantile) loss for quantile τ is used, where $e = Y - q_\tau$ is the error of the predicted quantile:

$$L_\tau(Y, q_\tau) = max(\tau \cdot e , (\tau - 1) \cdot e)$$

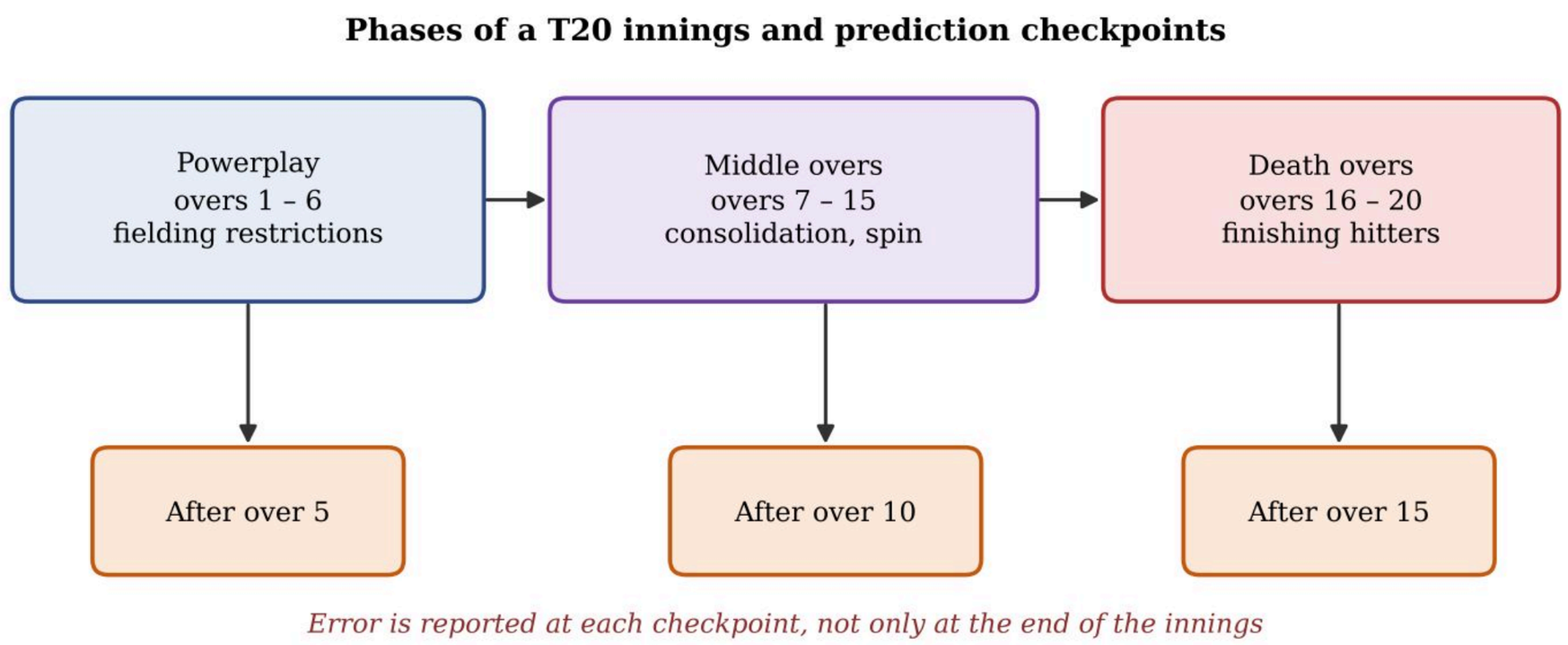


Figure 5.1: Phases of a T20 innings and the prediction checkpoints used for evaluation

5.2 Baseline Pipeline

1. **Data selection:** take ball-by-ball records of IPL matches and keep first-innings data.
2. **Aggregation:** convert the ball-by-ball records into over-level match states with

score, wickets and runs in each over.

3. **Run-rate baseline:** compute the CRR projection at each over.
4. **Resource baseline:** compute the resource-based projection using a resource table.
5. **Classical models:** train Linear Regression, Random Forest and XGBoost on the state features as in [2], [3].
6. **Decision:** compare the final-score error of every baseline at overs 5, 10 and 15 on the same test matches.

5.3 Proposed Context-Aware Framework

The proposed framework reads the innings as a short time series of overs. A recurrent encoder summarises how the innings has developed, learned embeddings represent the teams and the venue, and resource features give the model the wickets in hand and overs left. A small fusion head then outputs three quantiles of the final score. The design draws on the strengths identified in the survey: a sequence model as in [1] and [5], and the resource idea from [6] and [7].

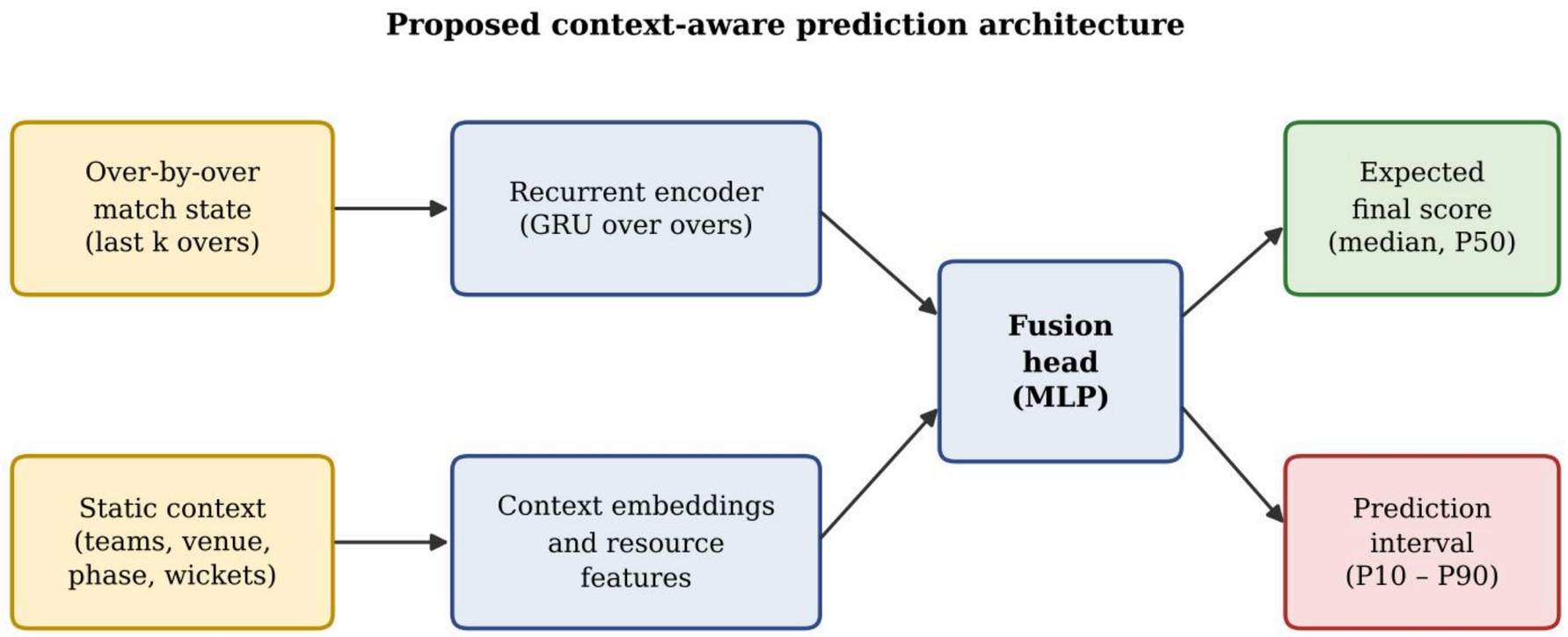


Figure 5.2: Proposed context-aware score prediction architecture

5.3.1 Input Features

- **Over-by-over history:** for each of the last k completed overs, the runs scored, wickets lost and boundaries hit.
- **Match state:** current score, wickets lost, overs completed and current run rate.
- **Resource features:** wickets in hand, overs remaining and a resource-remaining value in the spirit of [6], [7].
- **Context:** batting team, bowling team and venue, represented by learned embeddings, and optionally the toss decision.

5.3.2 Sequence Encoder

The over-level features are passed through a Gated Recurrent Unit (GRU) or LSTM layer. The final hidden state is a compact summary of the momentum of the innings. A GRU is chosen as the default because it has fewer parameters than an LSTM and trains quickly on a modest dataset, while the LSTM of [1] is kept as a

comparison.

5.3.3 Context and Fusion Head

The encoder output is concatenated with the team, venue and resource features and passed through a small Multi-Layer Perceptron. The head contains only a few thousand parameters, so its cost is negligible and a prediction can be produced in real time.

5.3.4 Uncertainty Head

The head outputs three values: the 10th, 50th and 90th percentiles of the final score. The middle value is the expected score; the outer two define an 80% prediction interval. Because the interval is learned from the data, it is wide early in the innings, when many outcomes are possible, and narrows as overs are completed. This addresses Gap 2 directly.

5.3.5 Training Objective

The model is trained with a Huber loss on the median forecast, which is less sensitive to rare extreme totals, combined with the pinball loss over the three quantiles weighted by a hyperparameter λ:

$$L = L_{Huber}(q_{50}, Y) + \lambda \cdot \Sigma_{\tau} L_{\tau}(Y, q_{\tau}), \quad \tau \in \{0.1, 0.5, 0.9\}$$

5.4 Data Collection and Preprocessing

Training and evaluation require ball-by-ball records with the final score of each innings. The source in [1] is a decade of IPL ball-by-ball data from ESPN and Cricsheet, and the same public sources are suitable here. Preprocessing includes the following steps.

- Parse each match file and keep first innings only; remove abandoned or shortened matches so that every innings has a 20-over target.
- Standardise team and venue names, since franchises and grounds are renamed over time (for example, Delhi Daredevils became Delhi Capitals).
- Build one training sample for every completed over of every innings, so that a single match supplies about twenty examples.
- Scale numeric features and encode categorical features as embeddings.
- Split the data by season into training, validation and test sets rather than at random. Random splitting would put overs from the same match in both sets, which leaks information and inflates the accuracy.

5.5 Model Training

- **Optimizer:** Adam with a small learning rate and a learning-rate scheduler.
- **Regularisation:** dropout and weight decay to prevent overfitting, since the number of independent matches is limited.
- **Hyperparameters:** history length k, hidden size, embedding size and λ, tuned on the validation seasons.
- **Early stopping:** based on validation MAE at overs 5, 10 and 15.

5.6 Evaluation Plan

Metric	Purpose
MAE and RMSE (runs)	Average and large-error behaviour of the point forecast, compared with the CRR and resource baselines and with the LSTM of [1].
R^2 score	Share of the variation in final scores that the model explains.
Accuracy within ±10 runs	Practical measure for captains and broadcasters, because a forecast within ten runs is useful in a T20 innings.
Error by stage (overs 5, 10, 15)	Shows how the error shrinks through the innings and where the model helps most over the baselines; the second research gap.
Interval coverage and width	The share of true scores that fall inside the P10–P90 interval, which should be close to 80%, and the average width of the interval.
Latency per prediction	Time for one forecast; the model should respond in a fraction of a second so that it can run live.

5.7 Challenges and Mitigation

- **Changing scoring rates:** T20 batting has become more aggressive over time [6]; training on recent seasons and testing on the latest season guards against this drift.
- **Rule changes:** changes such as the impact-player substitution in the IPL alter team composition; a season indicator or a recent-form feature can be added.
- **Limited independent matches:** only a few hundred matches per decade exist, so the model is kept small and regularised.
- **Venue and team sparsity:** rarely seen venues have little data; embeddings with dropout and a fallback to an average venue are used.
- **Data leakage:** features that use information from later overs must be excluded, and the split is by season.

5.8 Summary of Methodology

In summary, the methodology moves from a precise definition of the run-rate and resource baselines to a design that keeps the sequence-modelling strength of the LSTM in [1], adds the resource idea from [6] and [7], and replaces a single-number forecast with an expected score and an interval. The evaluation reports error at each stage of the innings, which makes the reliability of the model visible.

IMPLEMENTATION

This chapter describes how the baselines and the proposed model are to be implemented, together with an illustrative worked example. The pseudocode and numbers presented here illustrate the intended behaviour of the method; they are not reported experimental results.

6.1 Implementation Workflow

1. Download Cricsheet ball-by-ball data and load the first innings of every IPL match.
2. Aggregate balls into overs and build one feature row per completed over, with the final score of the innings as the label.
3. Split the rows by season into training, validation and test sets.
4. Compute the run-rate and resource baselines and train the classical regressors.
5. Train the GRU-based model with the quantile objective.
6. Evaluate MAE, RMSE, R^2, error by stage and interval coverage on the held-out season, comparing with the baselines.
7. Optionally wrap the model in a small Streamlit or Flask application for live use.

6.2 Baselines: Run Rate and Resources

The following Python-style sketch shows the two reference projections.

```
def crr_projection(score, overs_done, total_overs=20):
    return score / overs_done * total_overs


def resource_projection(score, resource_remaining):
    # resource_remaining: share of the innings resources still unused (0..1)
    return score / (1.0 - resource_remaining)
```

Both are computed from the current state alone. They are cheap, but they cannot learn from the data how a particular venue or phase behaves, which is what the proposed model adds.

6.3 Proposed Model: Sequence Encoder with Quantile Head

```
class ScoreNet(nn.Module):
    def __init__(self, d_over, n_teams, n_venues, d_emb=16, d_h=64):
        super().__init__()
        self.team = nn.Embedding(n_teams, d_emb)
        self.venue = nn.Embedding(n_venues, d_emb)
        self.gru = nn.GRU(d_over, d_h, batch_first=True)
        self.head = nn.Sequential(
            nn.Linear(d_h + 3 * d_emb + 3, 64), nn.ReLU(),
            nn.Dropout(0.2), nn.Linear(64, 3))      # P10, P50, P90

    def forward(self, overs, bat, bowl, venue, ctx):
        _, h = self.gru(overs)                          # overs: (B, k, d_over)
        z = torch.cat([h[-1], self.team(bat), self.team(bowl),
                       self.venue(venue), ctx], dim=-1)
        return self.head(z)                             # (B, 3)

def pinball(pred, y, taus=(0.1, 0.5, 0.9)):
    loss = 0.0
    for j, tau in enumerate(taus):
        e = y - pred[:, j]
```

```
        loss = loss + torch.maximum(tau * e, (tau - 1) * e).mean()
    return loss


# training step
pred = model(overs, bat, bowl, venue, ctx)
loss = F.huber_loss(pred[:, 1], y) + lam * pinball(pred, y)
loss.backward(); optimizer.step(); optimizer.zero_grad()
```

Here, ctx holds three numbers per sample: wickets in hand, overs remaining and the resource-remaining value. The model is small enough to train on a CPU and to give a prediction in a fraction of a second.

6.4 Illustrative Worked Example

Consider a first innings in which the team is 82 for 3 after 10 overs. The run-rate projection is 82 / 10 × 20 = 164. Suppose a resource table gives 55% of the innings resources remaining at this point (an illustrative value, not taken from an official table); then the resource projection is 82 / (1 − 0.55) ≈ 182. The two baselines differ by 18 runs, which shows how sensitive the forecast is to what is assumed about the remaining overs.

A trained context-aware model would instead use what has happened in the last overs, the venue and the teams. The table shows the kind of output expected; the numbers are illustrative.

Method	Prediction after 10 overs	Interval	Comment
Run rate (CRR × 20)	164	None	Assumes a constant scoring rate
Resource projection	182	None	Depends on the table used
Proposed model (illustrative)	176	162 – 190	Uses recent overs, venue and wickets; width reflects uncertainty
Actual final score (illustrative)	179	–	Falls inside the interval

Table 6.1: Illustrative comparison of predictions at the 10th over

6.5 Expected Behaviour of the Proposed Model

Scenario	Run-rate baseline	Proposed model
Steady innings with wickets in hand	Underestimates the final score because it ignores the death-overs acceleration	Wickets in hand and phase features allow a higher projection
Early collapse	Can over-project if the early overs were fast	Wickets lost and recent overs lower the projection
High-scoring venue	Not aware of the venue	Venue embedding shifts the expected total
Early in the innings (over 5)	Single number, no warning of uncertainty	Wide interval that narrows through the innings

The first and last rows are the central hypotheses of the project. Verifying them experimentally, by measuring error and interval coverage at overs 5, 10 and 15, is the

primary evaluation planned for the full implementation.

MAPPING TO SUSTAINABLE DEVELOPMENT GOAL 9

7.1 Overview of SDG 9

United Nations Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) aims to build resilient infrastructure, promote inclusive and sustainable industrialisation, and foster innovation. Sports technology, which includes live analytics, broadcasting graphics and performance analysis, is a growing industry in which data and machine learning are central. Reliable and affordable prediction tools strengthen the technological capability of that industry.

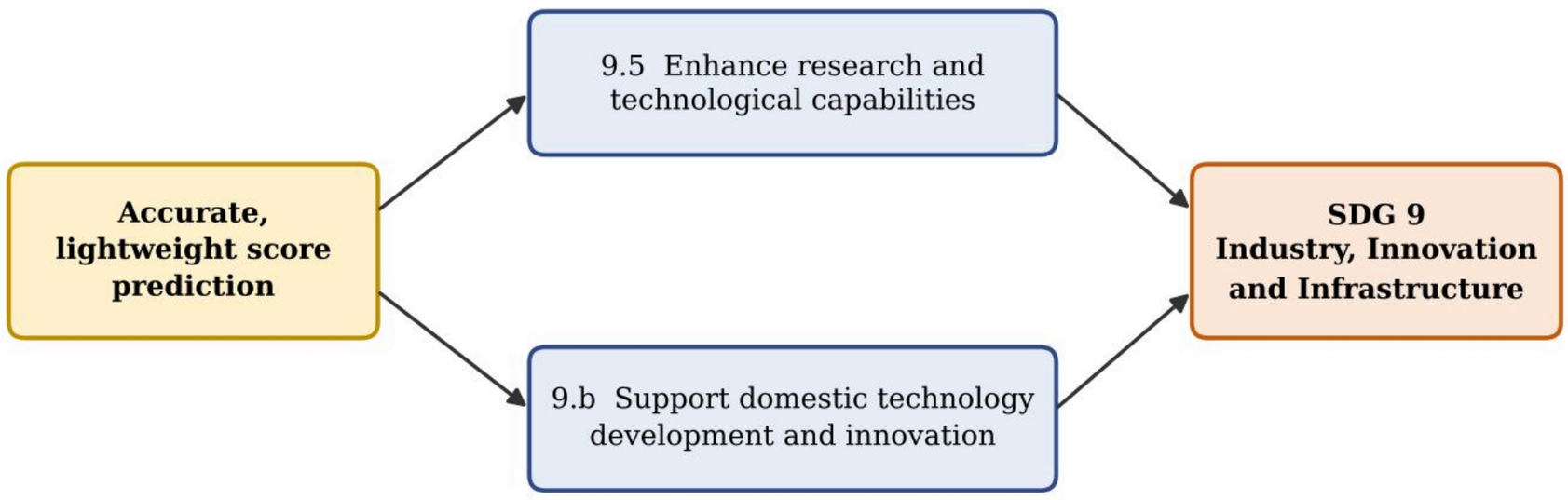


Figure 7.1: Contribution of cricket score prediction to SDG 9

7.2 How This Work Contributes

- **Technological capability (Target 9.5):** the project applies deep learning for sequential data to a real forecasting problem and documents the method, which builds research and technical skills.
- **Innovation in sports technology:** a model that gives scores with intervals supports better products for teams, broadcasters and analysts.
- **Affordable adoption (Target 9.b):** the proposed model is small and trains on public data, so institutions and start-ups that cannot afford large infrastructure can still build such tools.
- **Transparent and trustworthy AI:** reporting uncertainty and stage-wise error makes the system more honest about when it can be relied on.
- **Efficient resource use:** a compact model needs less computation and energy than a heavy model, supporting sustainable industrial innovation.

7.3 Summary

By addressing the score prediction research gap with a lightweight, transparent and reproducible framework, this project supports technological capability and innovation in the sports-analytics industry, in line with the objectives of SDG 9.

CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project surveyed the state of cricket score prediction, taking the deep learning study of IPL score prediction [1] as its foundation and comparing it with nine further studies spanning data-mining approaches, machine learning ensembles, deep learning for first-innings forecasting, resource-based methods and IPL outcome prediction. The foundational study showed that sequence-aware models, and the LSTM in particular, give the best accuracy on ball-by-ball IPL data, reaching an 85% success rate.

The analysis also revealed two limitations: forecasts that use little of the live match context, and a lack of prediction intervals and stage-wise evaluation. These define a clear research gap, which was formulated as a research question and answered with a proposed context-aware architecture that combines a recurrent encoder over the last overs, team, venue and resource features, and a quantile head that outputs an expected score together with an 80% interval.

8.2 Limitations of the Present Work

- The report is based on a literature survey; the proposed architecture is a design whose performance claims still need empirical validation.
- The worked example and the expected outputs are illustrative and are not experimental results.
- The survey relies on the published abstracts and summaries of the papers, and the results quoted from them depend on their own datasets and settings, so they are not directly comparable with each other.

8.3 Future Work

1. Implement the proposed model and benchmark it against the run-rate and resource baselines and the LSTM of [1] on error by stage and interval coverage.
2. Extend the model to the second innings by adding the target, required run rate and a win probability output.
3. Evaluate the transfer of the model to other T20 leagues and to ODI matches, building on [2] and [7].
4. Add player-level features such as the batters at the crease and the bowler type, in the spirit of [9] and [10].
5. Deploy the model as a small web application that updates the forecast after every over.

REFERENCES

[1] A. Raghav, N. Singh, and N. Kumar, “Sports Analytics in Cricket – Predicting Scores in the Indian Premier League Using Deep Learning Techniques,” ICTACT Journal on Data Science and Machine Learning, vol. 6, no. 3, pp. 808–815, Jun. 2025.

[2] T. Singh, V. Singla, and P. Bhatia, “Score and winning prediction in cricket through data mining,” in Proc. 2015 Int. Conf. on Soft Computing Techniques and

Implementations (ICSCTI), 2015, pp. 60–66.

[3] R. R. Kamble, N. Koul, K. Adhav, A. Dixit, and R. Pakhare, “Cricket Score Prediction Using Machine Learning,” Turkish Journal of Computer and Mathematics Education, vol. 12, no. 1S, pp. 23–28, 2021.

[4] R. Suguna, Y. Praveen Kumar, J. Suriya Prakash, P. S. Neethu, and S. Kiran, “Utilizing Machine Learning for Sport Data Analytics in Cricket: Score Prediction and Player Categorization,” in Proc. 2023 IEEE 3rd Mysore Sub Section Int. Conf. (MysuruCon), 2023.

[5] D. Abeysuriya, S. Fernando, and R. Navarathna, “Beyond the Run-rate: Forecasting Framework for First Innings Score in T20 Cricket,” in Proc. 2023 Moratuwa Engineering Research Conf. (MERCon), Moratuwa, Sri Lanka, 2023.

[6] R. Bhattacharya, P. S. Gill, and T. B. Swartz, “Duckworth–Lewis and Twenty20 cricket,” Journal of the Operational Research Society, vol. 62, no. 11, pp. 1951–1957, Nov. 2011.

[7] K. Abbas and S. Haider, “Duckworth-Lewis-Stern method comparison with machine learning approach,” conference paper, Institute of Business Administration, Karachi, Dec. 2019.

[8] K. C. Srikantaiah, A. Khetan, B. Kumar, D. Tolani, and H. Patel, “Prediction of IPL Match Outcome Using Machine Learning Techniques,” in Proc. 3rd Int. Conf. on Integrated Intelligent Computing Communication & Security, 2021; arXiv:2110.01395.

[9] S. Sanjaykumar, S. Natarajan, P. Y. Lakshmi, and F. A. Boby, “Predicting Team Success in the Indian Premier League Cricket 2024 Season Using Random Forest Analysis,” Physical Education Theory and Methodology, vol. 24, no. 2, pp. 304–309, 2024, doi: 10.17309/tmfv.2024.2.16.

[10] A. Menon, D. Khator, D. Prajapati, and A. Ekbote, “IPL Prediction Using Machine Learning,” Indian Journal of Computer Science, vol. 7, no. 3, pp. 23–29, 2022, doi: 10.17010/ijcs/2022/v7/i3/171267.